DEVIXO SOLUTIONS

AI/ML Internship Program

Task 02

Build and Compare Machine Learning Models

Student Name: Muhammad Ali

Internship Domain: AI/ML

Submission Date: October 18, 2026

Project: Customer Churn Classification and Monthly Charges Regression on the Telco Customer Churn Dataset

1. Introduction

This report documents Task 02 of the Devixo Solutions AI/ML Internship Program. It follows the full machine learning workflow: loading a real dataset, preprocessing it, training several models, evaluating them with suitable metrics, comparing them, tuning them with cross-validation (bonus) and recommending the best model with evidence.

The dataset describes the customers of a telecom company. Two problems are solved on it: **Task A (main)** is a classification problem, predicting whether a customer leaves (`Churn`); **Task B** is a regression problem, predicting the `MonthlyCharges` of a customer.

2. Objective

The objective is to learn why preprocessing, model comparison and model selection matter: to train at least three different algorithms, evaluate each with appropriate metrics, compare them on training time and performance, tune them, and choose a model on evidence instead of preference.

3. Dataset Name

Telco Customer Churn (file: `WA_Fn-UseC_-Telco-Customer-Churn.csv`).

4. Dataset Source

Kaggle: https://www.kaggle.com/datasets/blastchar/telco-customer-churn. It is an IBM sample dataset about a **fictional** telecom company that sells phone and internet services. Because the company is fictional, the findings describe this dataset and may not transfer to the customers of a real company. This is stated again in the limitations.

5. Dataset Description

Each row is one customer. The columns describe the customer (gender, senior citizen, partner, dependents), the account (tenure in months, contract, billing and payment method), the services the customer has (phone, internet, security, backup, device protection, tech support, streaming), the charges, and whether the customer left (`Churn`).

6. Dataset Size

	Raw dataset	After preprocessing
Rows (customers)	7,043	7,021
Columns	21	20 (`customerID` removed)
Blank `TotalCharges` values (hidden missing)	11	0 (filled with 0)
Duplicate rows (ignoring `customerID`)	22	0 (removed)
Customers who churned	1,869 (26.5%)	1,857 (26.4%)
File size	about 0.98 MB	-

7. Features/Columns Description

Group	Columns	Meaning
Identifier	customerID	Unique ID of the customer; dropped because it has no predictive meaning
Demographics	gender, SeniorCitizen, Partner, Dependents	Customer profile; `SeniorCitizen` is already 0/1
Account	tenure, Contract, PaperlessBilling, PaymentMethod	Months with the company; contract type (Month-to-month, One year, Two year); billing and payment method
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	Which services the customer has; `InternetService` is DSL, Fiber optic or No
Charges	MonthlyCharges, TotalCharges	Current monthly bill and total billed so far (`TotalCharges` is stored as text in the raw file)
Targets	Churn (Task A), MonthlyCharges (Task B)	`Churn` = Yes/No (encoded 1/0); `MonthlyCharges` is the regression target

8. Methodology

The work was done in Python 3 with Pandas, NumPy, Scikit-learn, Matplotlib and Seaborn in a Jupyter notebook (Google Colab). All scikit-learn models run on the CPU, so the Colab GPU was not needed. The random seed is 42 everywhere.

- **Part 1 - Preprocessing:** load, fix hidden missing values, remove duplicates, encode, scale and split (split first, then fit the preprocessing on the training data only).
- **Part 2 - Model development:** five algorithms for classification and five for regression, each with a Dummy model as a reference floor.
- **Part 3 - Evaluation:** Accuracy, Precision, Recall, F1 and confusion matrices (classification); MAE, MSE, RMSE and R2 (regression).
- **Bonus - Tuning:** `GridSearchCV` with 5-fold stratified cross-validation on the training set only, scored by F1.
- **Part 4 - Comparison:** tables with training time, metrics, advantages and limitations.
- **Part 5 - Conclusion:** a recommended model for each task and the reasons.

9. Part 1 - Data Preprocessing

Every action is recorded below with its reason. All preprocessing was done on a copy of the data; the raw file was never changed.

Step	Action	Affected	Reason
1	Loaded the CSV	7,043 rows, 21 columns	The task requires at least 1,000 records.
2	Converted `TotalCharges` from text to numbers and filled blanks with 0	11 values	`isnull()` found 0 missing values because the blanks are text. All 11 of them have `tenure` = 0, so these customers have not been billed yet and 0 is the logical value (a median would invent a bill).
3	Removed duplicate rows (ignoring `customerID`)	22 rows	Identical in every feature and in the label. Removing them also stops the same row from appearing in both the training and the test set. Assumption: they are redundant records.
4	Dropped `customerID`	1 column	A unique ID has no predictive meaning.
5	Encoded the target: No = 0, Yes = 1	`Churn`	Churn is the positive class, so Precision and Recall describe churners.
6	Split 80% / 20%, stratified, `random_state=42`	5,616 train / 1,405 test	Stratifying keeps the churn rate the same in both sets (0.2644 train, 0.2648 test).
7	Scaled `tenure`, `MonthlyCharges`, `TotalCharges` with `StandardScaler`	3 columns	Needed by Logistic Regression, KNN and SVM. Fitted on the training data only (inside a Pipeline) to avoid data leakage.
8	One-hot encoded the other 16 categorical columns	16 columns	Models need numbers. `handle_unknown="ignore"` keeps the pipeline safe for unseen categories.

Result: 7,021 clean rows (the requirement of at least 1,000 records is met), 20 columns before encoding, and no missing values or duplicates.

10. Exploratory Data Analysis

Statistics of the numeric columns (cleaned data, all rows):

Variable	Mean	Median	Std. dev.	Min	Max
tenure	32.47	29.00	24.53	0.00	72.00
MonthlyCharges	64.85	70.40	30.07	18.25	118.75
TotalCharges	2,286.77	1,400.55	2,266.86	0.00	8,684.80

Categories: contracts are Month-to-month (3,853), Two year (1,695) and One year (1,473); internet service is Fiber optic (3,090), DSL (2,419) and No internet (1,512).

The next figure uses the **training data only**, so no idea is formed by looking at the test set.

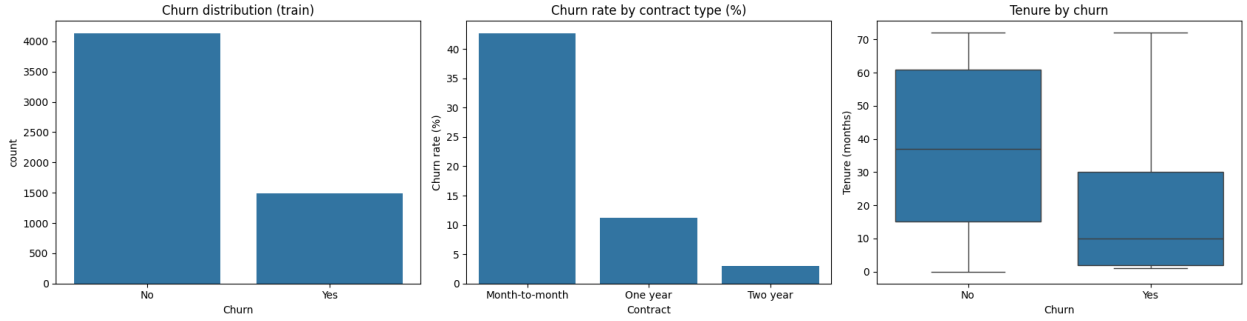


Figure 10.1: churn distribution, churn rate by contract type and tenure by churn (training set)

The churn rate is **42.7%** for month-to-month contracts, **11.2%** for one-year and **2.9%** for two-year contracts. The median tenure is **37 months** for customers who stayed and **10 months** for customers who left. Churn is the minority class (about 26% of customers), so a stratified split is used and accuracy cannot be the only metric. These are associations, not causes, and contract type and tenure are probably related to each other.

11. Part 2 - Model Development

Every model is a Scikit-learn `Pipeline` made of the shared preprocessing step and the model, trained on the same training set. Baseline models use default settings. A Dummy model that always predicts the majority class (classification) or the training mean (regression) is added as a reference floor.

Model	Task A (churn)	Task B (price)	Needs scaling?	Baseline settings
Logistic Regression	Yes	- (Linear Regression is used)	Yes	`max_iter=1000`
Linear Regression	-	Yes	Yes (helps)	defaults
Decision Tree	Yes	Yes	No	defaults (no depth limit)
Random Forest	Yes	Yes	No	defaults (100 trees)
KNN	Yes	Yes	Yes	defaults (5 neighbours)
SVM / SVR (RBF kernel)	Yes	Yes	Yes	defaults
Dummy	Majority class	Training mean	-	reference only

XGBoost (optional in the task) was not used, to keep every model explainable.

12. Part 3 - Model Evaluation (Task A, baseline models)

Metrics are calculated on the same test set for every model. The positive class is churn. Precision is the share of flagged customers who really churn; Recall is the share of real churners that are found; F1 balances the two. The training times in this first table were measured in single runs (a fairer timing is in section 13.3).

Model	Train time (s)	Accuracy	Precision	Recall	F1
Dummy (majority class)	0.074	0.7352	0.0000	0.0000	0.0000
Logistic Regression	0.182	0.8021	0.6610	0.5188	0.5813
Decision Tree	0.167	0.7139	0.4634	0.5108	0.4859
Random Forest	2.546	0.7751	0.5993	0.4543	0.5168
KNN	0.059	0.7594	0.5512	0.4919	0.5199
SVM (RBF)	1.595	0.7979	0.6618	0.4839	0.5590

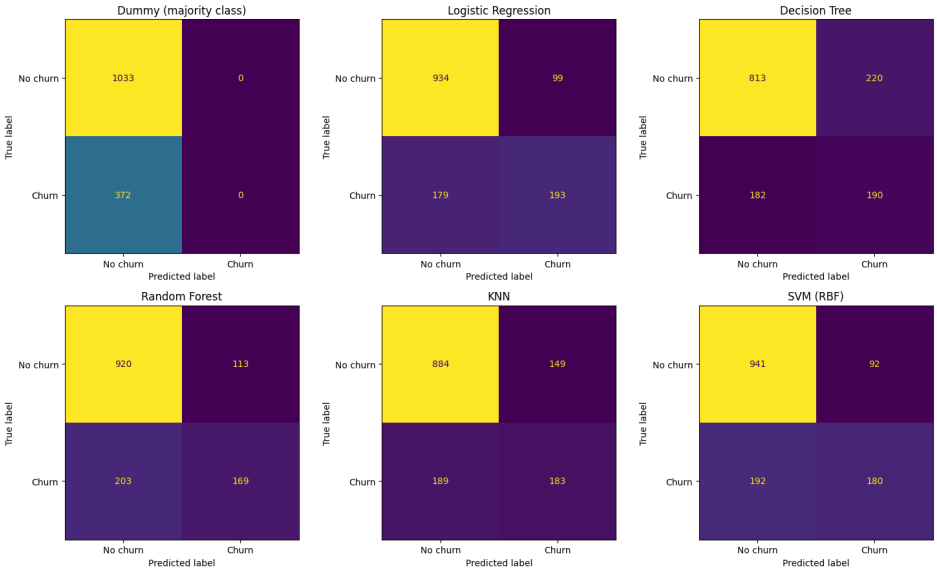


Figure 12.1: confusion matrices of the baseline models on the test set

- The Dummy model has accuracy **0.7352**, which is the share of non-churners in the test set. All five real models beat it on F1.
- Logistic Regression has the best baseline accuracy (**0.8021**) and F1 (**0.5813**).
- The unpruned Decision Tree has accuracy **0.7139**, below the Dummy, but its recall (0.5108) is higher than KNN, SVM and Random Forest. Accuracy and recall tell different stories, so accuracy alone is misleading here.
- Every baseline model has recall between **0.4543 and 0.5188**: each finds only about half of the 372 churners in the test set. Churners missed: Random Forest 203, SVM 192, KNN 189, Decision Tree 182, Logistic Regression 179.

13. Bonus - Hyperparameter Tuning (GridSearchCV)

Design. `GridSearchCV` with small grids and **5-fold stratified cross-validation on the training set only**, scored by **F1** of the churn class (accuracy rewards predicting 'No' on imbalanced data). `class_weight` is part of the grid for the models that support it, because it makes a model pay more attention to the rare churn class. After the search, each model is refitted with the best settings and evaluated **once** on the test set, so the test set never influences the choice of settings.

13.1 Grids searched

Model	Parameters searched
Logistic Regression	`C` in {0.01, 0.1, 1, 10}; `class_weight` in {None, balanced}
Decision Tree	`max_depth` in {3, 5, 7, 10}; `min_samples_leaf` in {1, 10, 30}; `class_weight` in {None, balanced}
Random Forest	`n_estimators` in {100, 300}; `max_depth` in {5, 10, None}; `min_samples_leaf` in {1, 5, 10}; `class_weight` in {None, balanced}
KNN	`n_neighbors` in {5, 11, 21, 31}; `weights` in {uniform, distance}
SVM (RBF)	`C` in {0.1, 1, 10}; `gamma` in {scale, 0.01}; `class_weight` in {None, balanced}

13.2 Results and before/after comparison

Model	Best parameters	CV F1 (train)	Accuracy	Precision	Recall	F1	Search (s)
Logistic Regression	{'C': 10, 'class_weight': 'balanced'}	0.6299	0.7381	0.5035	0.7823	0.6126	5.3
Decision Tree	{'class_weight': 'balanced', 'max_depth': 5, 'min_samples_leaf': 30}	0.6256	0.7423	0.5091	0.7527	0.6074	9.2
Random Forest	{'class_weight': 'balanced', 'max_depth': 10, 'min_samples_leaf': 10, 'n_estimators': 300}	0.6397	0.7658	0.5416	0.7527	0.6299	173.8
KNN	{'n_neighbors': 31, 'weights': 'uniform'}	0.6056	0.7872	0.6137	0.5296	0.5685	3.5
SVM (RBF)	{'C': 1, 'class_weight': 'balanced', 'gamma': 'scale'}	0.6242	0.7601	0.5325	0.7715	0.6301	74.1

Model	Stage	Accuracy	Precision	Recall	F1
Logistic Regression	Baseline	0.8021	0.6610	0.5188	0.5813
	Tuned	0.7381	0.5035	0.7823	0.6126
Decision Tree	Baseline	0.7139	0.4634	0.5108	0.4859
	Tuned	0.7423	0.5091	0.7527	0.6074
Random Forest	Baseline	0.7751	0.5993	0.4543	0.5168
	Tuned	0.7658	0.5416	0.7527	0.6299
KNN	Baseline	0.7594	0.5512	0.4919	0.5199
	Tuned	0.7872	0.6137	0.5296	0.5685
SVM (RBF)	Baseline	0.7979	0.6618	0.4839	0.5590
	Tuned	0.7601	0.5325	0.7715	0.6301

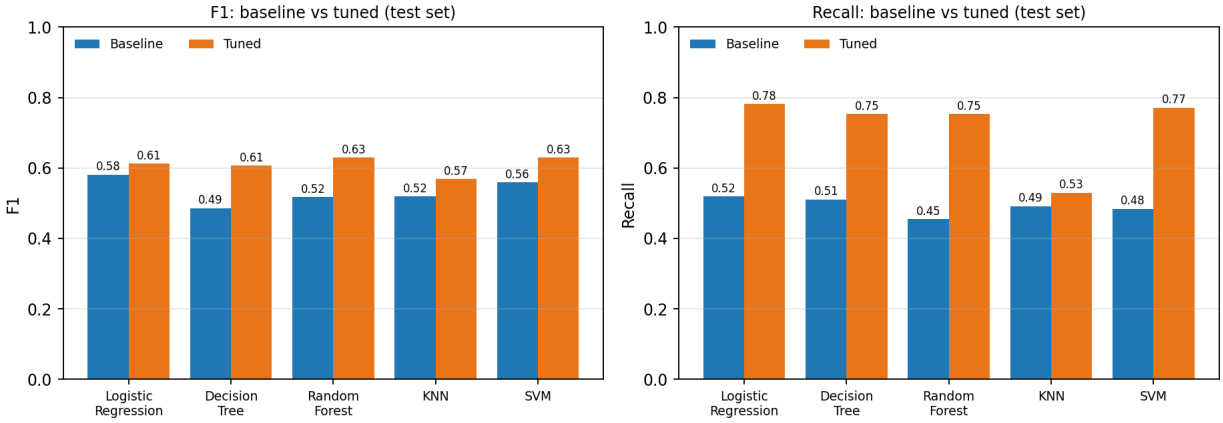


Figure 13.1: F1 and Recall before and after tuning (test set; re-drawn from the notebook's numbers)

- **Recall rose for every model:** Logistic Regression +0.2635, Decision Tree +0.2419, Random Forest +0.2984, KNN +0.0377, SVM (RBF) +0.2876.
- **F1 rose for every model:** Logistic Regression +0.0313, Decision Tree +0.1215, Random Forest +0.1131, KNN +0.0486, SVM (RBF) +0.0711.
- **Precision fell** for Logistic Regression, Random Forest, SVM (RBF) and rose for the others. **Accuracy fell** for Logistic Regression, Random Forest, SVM (RBF).
- `class_weight="balanced"` was chosen by all four models that support it. The Random Forest search took the longest (173.8 s), then SVM (74.1 s).
- Cross-validated F1 on the training data is close to the test F1 (largest gap 0.0371, for KNN), so there is no sign of serious overfitting.

- Tuned Logistic Regression has accuracy 0.7381, barely above the Dummy (0.7352), yet it finds 78.2% of the churners where the Dummy finds none.

Interpretation. The models trade precision for recall. With balanced class weights they flag more customers as churners, so they catch more real churners but also raise more false alarms. That is a business trade-off (a missed churner against a wasted retention offer), not a defect. KNN has no `class_weight` option, which is why it changed the least. Several winning values sit at the edge of their grid (Logistic Regression `C=10`, Decision Tree `min_samples_leaf=30`, Random Forest `n_estimators=300` and `min_samples_leaf=10`, KNN `n_neighbors=31`), so a wider grid might do slightly better.

13.3 Fair training-time comparison

The first timing tables are not fair: the baseline Random Forest used one CPU core and the tuned one used all cores, with 300 trees instead of 100. The times below refit every model with the **same CPU usage (one core)** and report the **median of 5 fits** (Colab CPU). The search time is the cost of the grid search itself (all cores).

Model	Baseline fit (s)	Tuned fit (s)	Search time (s)
Logistic Regression	0.077	0.114	5.3
Decision Tree	0.119	0.082	9.2
Random Forest	0.834	1.632	173.8
KNN	0.059	0.055	3.5
SVM (RBF)	1.329	1.737	74.1

The tuned Random Forest is about **2.0 times slower** to fit than the baseline, which matches 300 trees instead of 100. KNN is the fastest model to fit in both stages; SVM and Random Forest are the slowest. Differences of about 0.05 s are within the noise of a shared Colab machine and should not be interpreted.

13.4 Confusion matrices after tuning

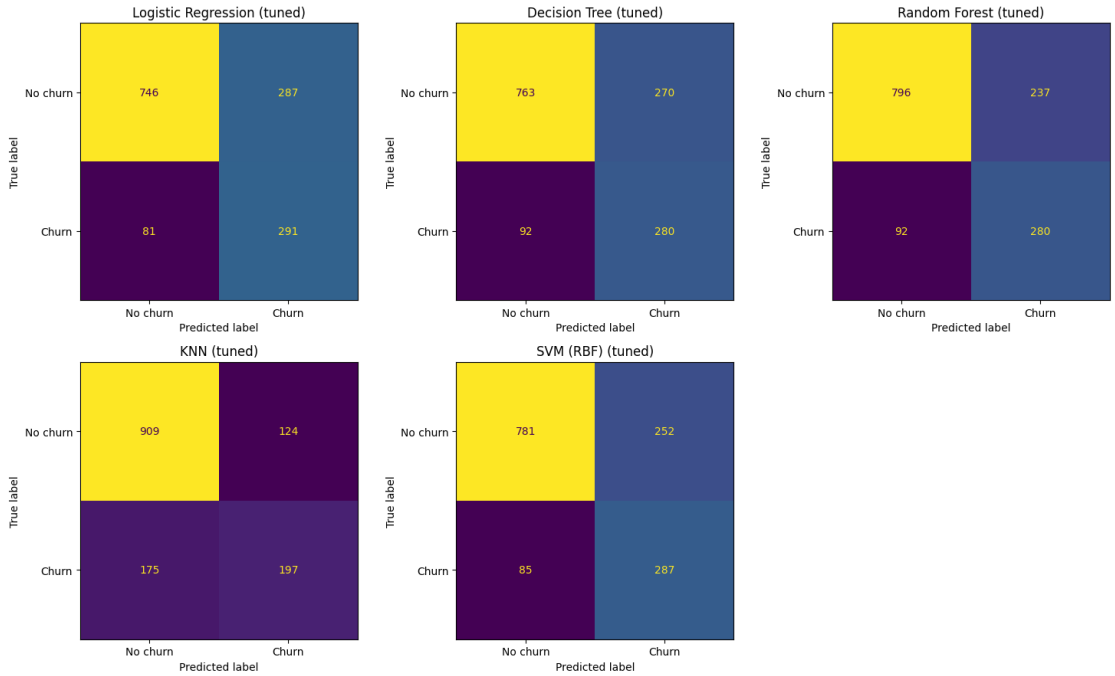


Figure 13.2: confusion matrices of the tuned models on the test set

Model	Churners found (TP)	Churners missed (FN)	False alarms (FP)
Logistic Regression	193 to 291	179 to 81	99 to 287
Decision Tree	190 to 280	182 to 92	220 to 270
Random Forest	169 to 280	203 to 92	113 to 237
KNN	183 to 197	189 to 175	149 to 124
SVM (RBF)	180 to 287	192 to 85	92 to 252

Counts are baseline to tuned. The test set has 372 churners and 1033 non-churners. After tuning, the models miss far fewer churners, at the price of more false alarms. The counts are derived from Precision and Recall and match the confusion-matrix charts.

14. Feature Importance (tuned Random Forest)

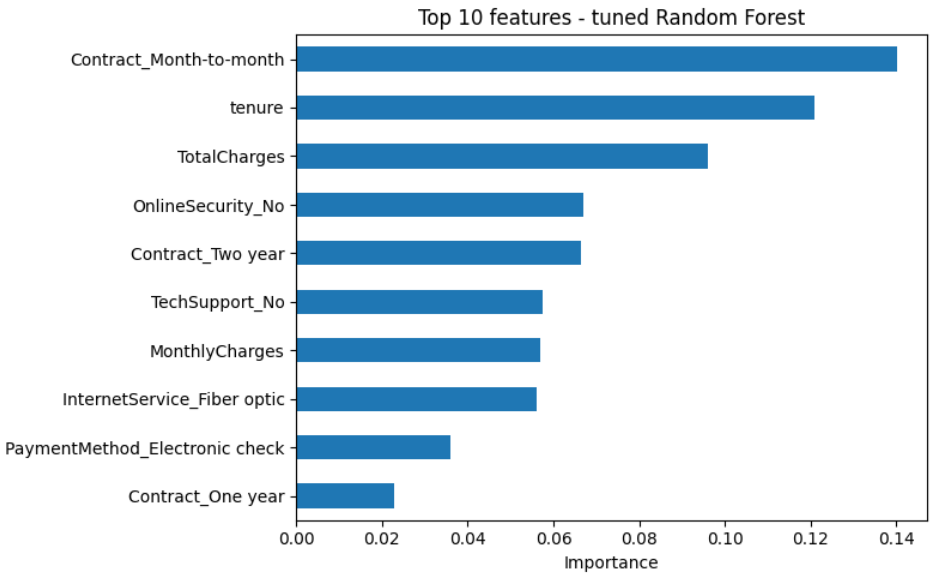


Figure 14.1: ten most important features (impurity-based importance; one-hot columns appear as Feature_Category)

Feature	Importance
Contract_Month-to-month	0.1402
tenure	0.1208
TotalCharges	0.0961
OnlineSecurity_No	0.0670
Contract_Two year	0.0664
TechSupport_No	0.0576
MonthlyCharges	0.0568
InternetService_Fiber optic	0.0562
PaymentMethod_Electronic check	0.0359
Contract_One year	0.0230

The most important features are `Contract_Month-to-month` (0.1402), `tenure` (0.1208) and `TotalCharges` (0.0961). 3 of the top ten columns come from `Contract`. This agrees with the exploratory analysis. Two cautions: the importance shows what the model **used**, not what **causes** churn; and impurity-based importance tends to favour numeric features with many distinct values, while a category such as Contract is split over its dummy columns.

15. Task B - Regression: Predicting MonthlyCharges

The same dataset also supports an honest regression question: the monthly price should depend on which services a customer has. **Target:** `MonthlyCharges`. **Excluded features:** `TotalCharges` (roughly `tenure` times `MonthlyCharges`, so it would leak the answer) and `Churn` (an outcome, not a pricing input). **Features kept:** `tenure` (scaled) and the 16 categorical columns (one-hot encoded). The split is 80/20 and not stratified (5,616 training rows, 1,405 test rows, seed 42).

Metrics: MAE is the average size of the error; MSE averages the squared errors; RMSE is the square root of MSE (in the units of the price, punishing large errors more); R2 is the share of the variation in price that the model explains (1 is perfect, 0 is no better than the mean).

Validation check. Training prices range from 18.25 to 118.75 (mean 64.83). The average price depends strongly on the internet service, but a large spread remains inside each type, so other services also change the price:

Internet service	Customers	Mean price	Std. dev.	Min	Max
No	1,512	21.09	2.17	18.25	26.90
DSL	2,419	58.11	16.26	23.45	94.80
Fiber optic	3,090	91.54	12.64	67.75	118.75

Regression results on the test set (default settings):

Model	Train time (s)	MAE	MSE	RMSE	R2
Dummy (mean)	0.033	25.7137	880.8155	29.6785	-0.0000
Linear Regression	0.056	0.7997	1.1061	1.0517	0.9987
Decision Tree	0.108	1.2426	2.8300	1.6823	0.9968
Random Forest	2.819	1.0069	1.8373	1.3555	0.9979
KNN	0.034	2.7637	17.7300	4.2107	0.9799
SVR (RBF)	2.297	1.0294	1.8587	1.3633	0.9979

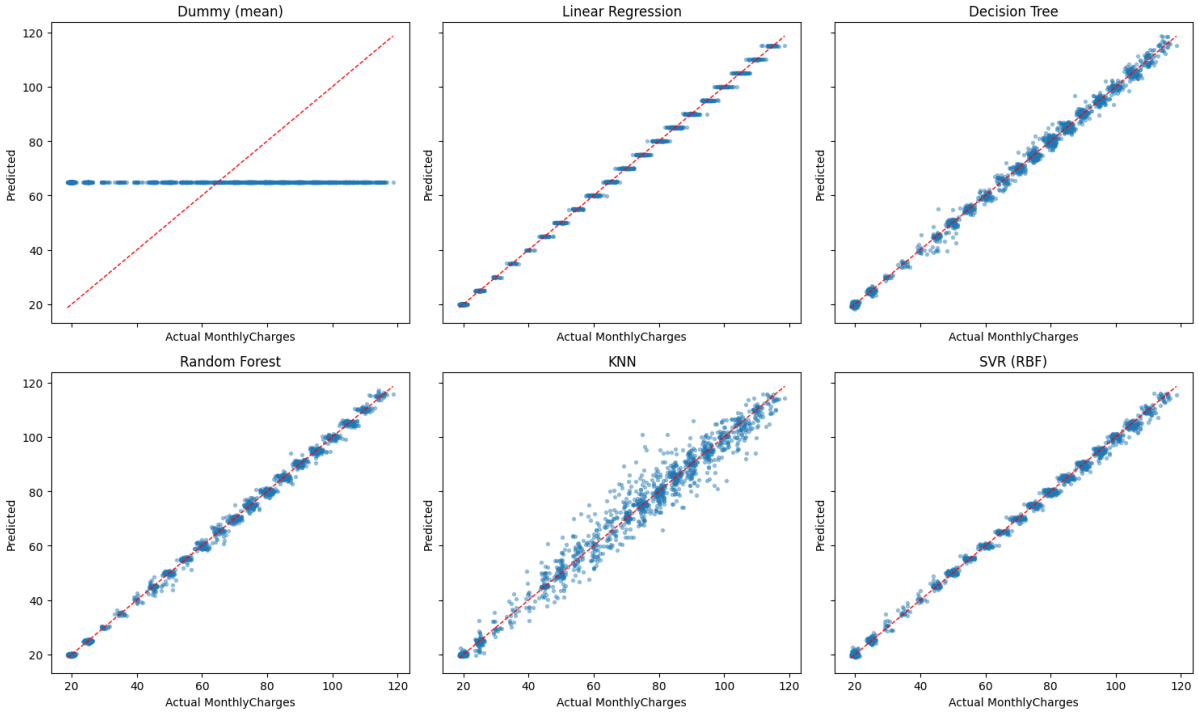


Figure 15.1: actual vs predicted MonthlyCharges on the test set (points near the red line are good predictions)

- The Dummy model has R2 = -0.0000 and MAE = 25.7137, so the metric calculation is wired correctly.
- **Linear Regression is best on all four metrics:** MAE 0.7997, MSE 1.1061, RMSE 1.0517, R2 0.9987.
- Random Forest and SVR both have R2 0.9979, the Decision Tree 0.9968, and KNN is the weakest real model (R2 0.9799, MAE 2.7637).

Interpretation (a hypothesis, not tested directly). The price looks close to a sum of fixed charges for each service. If so, a linear model fits it almost perfectly and the flexible models have nothing extra to find. The table above and the fact that Linear Regression beats the flexible models agree with this idea, but they do not prove it. KNN probably suffers because distances are noisy across many one-hot columns. Every real model has R2 above 0.97, so this task is **easy** and the comparison is less informative than for churn. No tuning was done for regression; the bonus requirement is covered by Task A.

16. Part 4 - Model Comparison

Classification (Task A). The table shows the final, tuned version of each model on the test set. Fit time is the median of 5 single-core fits; the search time is the cost of the grid search. The baseline results are in section 12 and the before/after comparison is in section 13.2.

Model	Fit time (s)	Acc.	Prec.	Recall	F1	Advantages	Limitations
Logistic Regression	0.114 (search 5.3)	0.7381	0.5035	0.7823	0.6126	Very fast to train, simple, easy to explain; highest recall after tuning (0.7823)	Assumes straight-line effects; tuned accuracy (0.7381) is barely above the 'always No' accuracy (0.7352)
Decision Tree	0.082 (search 9.2)	0.7423	0.5091	0.7527	0.6074	Easy to visualise and explain	Unpruned baseline had the lowest baseline F1 (0.4859); needs depth limits
Random Forest	1.632 (search 173.8)	0.7658	0.5416	0.7527	0.6299	Handles non-linear effects, gives feature importance; best CV F1 (0.6397)	Longest tuning search (173.8s); slower to fit than Logistic Regression, KNN or Decision Tree; harder to explain
KNN	0.055 (search 3.5)	0.7872	0.6137	0.5296	0.5685	Simple and the fastest to fit here (it mostly stores the data)	No class_weight option, so recall stayed low (0.5296); sensitive to scaling and many one-hot columns
SVM (RBF)	1.737 (search 74.1)	0.7601	0.5325	0.7715	0.6301	Handles non-linear boundaries; tied for best F1 (0.6301)	Second-longest search (74.1s) and slowest final fit; no built-in feature importance

Regression (Task B). Results on the test set with default settings (no tuning); the Dummy row is the reference floor.

Model	Train time (s)	MAE	RMSE	R2	Advantages	Limitations
Dummy (mean)	0.033	25.7137	29.6785	-0.0000	Reference point only	Ignores all features
Linear Regression	0.056	0.7997	1.0517	0.9987	Fast, simple, best on every metric here	Assumes additive effects; would miss complex interactions
Decision Tree	0.108	1.2426	1.6823	0.9968	Easy to explain, no scaling needed	Coarse predictions (steps); lower R2 than the top models
Random Forest	2.819	1.0069	1.3555	0.9979	Handles non-linear effects, stable	Slower than Linear Regression with no accuracy gain here
KNN	0.034	2.7637	4.2107	0.9799	Simple, no assumptions about shape	Worst R2 and MAE of the real models; distance gets noisy with many one-hot columns
SVR (RBF)	2.297	1.0294	1.3633	0.9979	Flexible, accurate (R2 0.9979)	Slower; needs scaled inputs; hard to interpret

17. Part 5 - Conclusion and Recommendation

I trained and compared five classification models and five regression models on the Telco Customer Churn dataset (7,021 customers after cleaning). I fixed hidden missing values, removed 22 duplicate rows, split the data before fitting any scaler or encoder, and compared every model on the same test set. Because only about 26% of customers churn, I judged the churn models by Precision, Recall and F1 and not by accuracy alone.

Classification: I recommend the tuned Random Forest (`n_estimators=300`, `max_depth=10`, `min_samples_leaf=10`, `class_weight=balanced`). Its test F1 (0.6299) is tied for the best with SVM (0.6301); the difference of 0.0002 is far smaller than the noise to expect from 372 test churners, so the two cannot be separated on F1. The tie is broken by three points: the Random Forest has the highest cross-validated F1 on the training data (0.6397), which is a steadier estimate than a single test split; among the models with a recall of about 0.75 or more it has the best precision (0.5416); and it provides feature importance. It finds 280 of the 372 churners, against 169 for the baseline forest. A likely reason it did better is that a forest can combine several features at once (for example contract type, tenure and services) while a linear model adds up separate effects; the small margin over Logistic Regression suggests that most of the pattern is simple. I did not test this explanation directly.

An honest alternative is **Logistic Regression**: its F1 is 0.0173 lower (0.6126), but it has the highest recall (0.7823), is easy to explain and fits more than ten times faster (0.114 s against 1.632 s). If the business prefers a simple, explainable model, it is a defensible choice. The right choice also depends on the cost of a missed churner against a false alarm, which this project did not model.

Regression: I recommend Linear Regression. It has the lowest MAE (0.7997), MSE (1.1061) and RMSE (1.0517) and the highest R2 (0.9987), and it is simple and fast. The likely reason is that the monthly price is close to an additive sum of service charges (a hypothesis supported by the results, not proven).

From this project I learned that `isnull()` can hide missing data, that accuracy is a poor metric on imbalanced data, that scaling and encoding must be fitted on training data only, that tuning can raise recall mainly by giving up precision, and that training times must be measured under equal conditions before they are compared.

18. Limitations

- **Fictional data:** the dataset describes a fictional telecom company, so the findings are for this dataset and not for real customers.
- **One train/test split** with only 372 churners in the test set: differences of about 0.01 to 0.02 in F1 are within noise. Cross-validated scores were used to support the choice.
- **Fixed 0.5 decision threshold and no cost model:** the business cost of a missed churner against a false alarm was not used.
- **Class weights trade precision for recall:** accuracy fell for 3 of the five tuned models, and tuned Logistic Regression's accuracy (0.7381) is barely above 'always No' (0.7352).
- **Small grids with best values at the grid edge:** a wider search might improve the scores slightly.
- **Timing on Colab is noisy:** only large differences are meaningful. Search times use all cores; fit times use one core.
- **Assumptions in preprocessing:** identical rows are redundant, and customers who have not been billed yet owe nothing (`TotalCharges` = 0).
- **Task B is easy:** all real models have R2 above 0.97, and no regression tuning was done.
- **Feature importance** is impurity-based and not causal. XGBoost (optional in the task) was not included.

19. Major Findings

- After cleaning, the dataset has 7,021 customers; 26.4% churned.
- Churn is much higher on month-to-month contracts (42.7%) than on two-year contracts (2.9%), and churners have a much shorter median tenure (10 against 37 months). These are associations, not causes.
- Default models find only 45.4% to 51.9% of the churners; accuracy hides this because the Dummy model already scores 0.7352.
- Tuning with balanced class weights raised recall for every model (the best reaches 0.7823) and F1 for every model, but lowered precision for 3 of five.
- The tuned Random Forest and SVM are tied on F1 (0.6299 and 0.6301); the forest is recommended for its cross-validated F1, precision and feature importance.
- `MonthlyCharges` can be predicted almost perfectly from the services (Linear Regression R2 0.9987), which suggests an additive price structure.

20. Appendix - Key Code from the Notebook

The code below is copied from the notebook cells (text, not screenshots). The notebook in the repository contains all cells with their saved outputs.

A.1 Data loading, cleaning and split (Part 1)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

RANDOM_STATE = 42
# The CSV is searched in: Colab upload, repo layout (notebook/ folder), repo root, current folder.
from pathlib import Path
_CANDIDATES = [Path("/content/WA_Fn-UseC_-Telco-Customer-Churn.csv"),
               Path("../dataset/WA_Fn-UseC_-Telco-Customer-Churn.csv"),
               Path("dataset/WA_Fn-UseC_-Telco-Customer-Churn.csv"),
               Path("WA_Fn-UseC_-Telco-Customer-Churn.csv")]
DATA_PATH = next((p for p in _CANDIDATES if p.exists()), None)
if DATA_PATH is None:
    raise FileNotFoundError("Dataset CSV not found. Upload it to Colab or put it in dataset/. Current folder: " + str(Path.cwd()))

# ---- 1. Load ----
df = pd.read_csv(DATA_PATH)
print("Raw shape:", df.shape)

# ---- 2. Fix TotalCharges (text -> number) ----
# Blank strings can't be converted, so they become NaN.
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Safety check: only the 11 known blanks, and all must be tenure == 0
missing_mask = df["TotalCharges"].isna()
assert missing_mask.sum() == 11, "Unexpected number of missing TotalCharges"
assert (df.loc[missing_mask, "tenure"] == 0).all(), "Missing TotalCharges with tenure > 0"

# New customers (tenure 0) have not been billed yet -> 0 is the logical value
df["TotalCharges"] = df["TotalCharges"].fillna(0)

# ---- 3. Remove duplicates (ignoring the unique ID) ----
n_before = len(df)
df = df.drop(columns="customerID")
df = df.drop_duplicates().reset_index(drop=True)
print(f"Removed {n_before - len(df)} duplicate rows. Clean shape: {df.shape}")
assert len(df) >= 1000

# ---- 4. Encode target ----
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})
assert df["Churn"].isna().sum() == 0

# ---- 5. Features / target ----
X = df.drop(columns="Churn")
y = df["Churn"]

# ---- 6. Split FIRST, stratified so both sets keep the same churn rate ----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE
)
print("Train:", X_train.shape, "| Test:", X_test.shape)
print("Churn rate  train:", round(y_train.mean(), 4), "| test:", round(y_test.mean(), 4))

# ---- 7. Preprocessor (fitted later, inside a Pipeline, on training data only) ----
numeric_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
categorical_cols = [c for c in X.columns if c not in numeric_cols]

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
])
```

Listing A.1: preprocessing cell

A.2 Baseline models and evaluation (Parts 2 and 3)

```
import time
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay)

models = {
    "Dummy (majority class)": DummyClassifier(strategy="most_frequent"),
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=RANDOM_STATE),
    "Decision Tree": DecisionTreeClassifier(random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(random_state=RANDOM_STATE),
    "KNN": KNeighborsClassifier(),
    "SVM (RBF)": SVC(random_state=RANDOM_STATE),
}

rows, conf_mats = [], {}
for name, model in models.items():
    # Pipeline = preprocessing + model, so scaling/encoding is fitted on train data only
    pipe = Pipeline([("prep", preprocessor), ("model", model)])

    start = time.perf_counter()
    pipe.fit(X_train, y_train)
    train_time = time.perf_counter() - start      # measured, not estimated

    preds = pipe.predict(X_test)                  # same test set for every model
    rows.append({
        "Model": name,
        "Train time (s)": round(train_time, 3),
        "Accuracy": accuracy_score(y_test, preds),
        "Precision": precision_score(y_test, preds, zero_division=0),
        "Recall": recall_score(y_test, preds),
        "F1": f1_score(y_test, preds),
    })
    conf_mats[name] = confusion_matrix(y_test, preds)

baseline_results = pd.DataFrame(rows).round(4)
baseline_results.to_csv("../results/baseline_classification_results.csv", index=False)
display(baseline_results)

# Confusion matrices
fig, axes = plt.subplots(2, 3, figsize=(15, 9))
for ax, (name, cm) in zip(axes.ravel(), conf_mats.items()):
    ConfusionMatrixDisplay(cm, display_labels=["No churn", "Churn"]).plot(ax=ax, colorbar=False)
    ax.set_title(name)
plt.tight_layout()
plt.savefig("../images/baseline_confusion_matrices.png", dpi=150)
plt.show()
```

Listing A.2: baseline models

A.3 Hyperparameter tuning (Bonus)

```
from sklearn.model_selection import StratifiedKFold, GridSearchCV

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

search_space = {
    "Logistic Regression": (
        LogisticRegression(max_iter=1000, random_state=RANDOM_STATE),
        {"model__C": [0.01, 0.1, 1, 10],
         "model__class_weight": [None, "balanced"]}),
    "Decision Tree": (
        DecisionTreeClassifier(random_state=RANDOM_STATE),
        {"model__max_depth": [3, 5, 7, 10],
         "model__min_samples_leaf": [1, 10, 30],
         "model__class_weight": [None, "balanced"]}),
    "Random Forest": (
        RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1),
        {"model__n_estimators": [100, 300],
         "model__max_depth": [5, 10, None],
         "model__min_samples_leaf": [1, 5, 10],
         "model__class_weight": [None, "balanced"]}),
    "KNN": (
        KNeighborsClassifier(),
        {"model__n_neighbors": [5, 11, 21, 31],
         "model__weights": ["uniform", "distance"]}),
    "SVM (RBF)": (
        SVC(random_state=RANDOM_STATE),
        {"model__C": [0.1, 1, 10],
         "model__gamma": ["scale", 0.01],
         "model__class_weight": [None, "balanced"]}),
}

tuned_rows, tuned_conf_mats, best_models = [], {}, {}
for name, (estimator, grid) in search_space.items():
    # "model__" prefix = parameter belongs to the "model" step of the pipeline
    pipe = Pipeline([("prep", preprocessor), ("model", estimator)])
    gs = GridSearchCV(pipe, grid, scoring="f1", cv=cv, n_jobs=-1, refit=True)

    start = time.perf_counter()
    gs.fit(X_train, y_train)                # search uses TRAIN data only
    search_time = time.perf_counter() - start

    preds = gs.predict(X_test)              # test set used once, after tuning
    tuned_rows.append({
        "Model": name,
        "Search time (s)": round(search_time, 1),
        "Final fit time (s)": round(gs.refit_time_, 3),
        "CV F1 (train)": gs.best_score_,
        "Accuracy": accuracy_score(y_test, preds),
        "Precision": precision_score(y_test, preds),
        "Recall": recall_score(y_test, preds),
        "F1": f1_score(y_test, preds),
        "Best params": str(gs.best_params_),
    })
    tuned_conf_mats[name] = confusion_matrix(y_test, preds)
    best_models[name] = gs.best_estimator_
    print(f"{name}: done in {search_time:.1f}s -> {gs.best_params_}")

tuned_results = pd.DataFrame(tuned_rows).round(4)
tuned_results.to_csv("../results/tuned_classification_results.csv", index=False)
display(tuned_results)
```

Listing A.3: GridSearchCV for all five models

A.4 Fair training-time measurement

```
from sklearn.base import clone

def median_fit_time(pipe, n_runs=5):
    times = []
    for _ in range(n_runs):
        p = clone(pipe)
        if "n_jobs" in p.named_steps["model"].get_params():
            p.set_params(model__n_jobs=1)          # same CPU usage for every model
        start = time.perf_counter()
        p.fit(X_train, y_train)
        times.append(time.perf_counter() - start)
    return round(float(np.median(times)), 3)

timing = {}
for name in best_models:
    base_pipe = Pipeline([("prep", preprocessor), ("model", models[name])])
    timing[name] = {"Baseline": median_fit_time(base_pipe),
                    "Tuned": median_fit_time(best_models[name])}

# Replace the old, unfair times in the comparison table
for name, t in timing.items():
    for stage in ["Baseline", "Tuned"]:
        mask = (comparison["Model"] == name) & (comparison["Stage"] == stage)
        comparison.loc[mask, "Fit time (s)"] = t[stage]

comparison.to_csv("../results/before_after_comparison.csv", index=False)
display(comparison)
```

Listing A.4: median of 5 single-core fits

21. GitHub Repository Link

https://github.com/Al7bh/Devixo-Internship (folder: `Task_02_ML`)

The task folder contains the notebook with saved outputs, the dataset folder (or download link), the figures in `images/`, the result tables in `results/`, this report, the report builder script, `requirements.txt` and the README.